Mayo Clinic – Digital Product Engineering Test 3

Project: Rules-Driven Decision Engine & Release Gate Simulator
Date: March 4, 2026
Candidate: Hameem Mahdi M.S.E., Ph.D.
Position Title: (374365) #25 – Software Engineer - Platform Engineering
Language: Node.js (zero external dependencies)
Entry point: `TEST 3/engineering_test_3/engine.js`

---

1. Overview

This project implements a fully deterministic, rules-driven clinical decision engine paired with a release gate simulator for a healthcare platform. Given a dataset of 100 synthetic patient cases, a security scan, load test results, canary deployment metrics, and a YAML-based policy configuration, the engine evaluates every required gate and produces structured machine-readable JSON outputs covering all nine sections of the specification.

The solution is implemented in a single Node.js file (`engine.js`) with no npm dependencies, using only Node's built-in `fs` , `path` , and `crypto` modules.

---

2. How to Run

```
cd "TEST 3/engineering_test_3"
node engine.js
```

All 12 output files are written to `healthcare_100/outputs/` . A 13th file (`decision_mismatches.jsonl`) is written only if mismatches are detected, per the specification.

---

3. Architecture

The engine reads all inputs at startup, applies processing in section order, and writes outputs after each section completes. Rule evaluation uses a priority-ordered list evaluated with first-match-wins semantics, exactly mirroring the CSV priority column.

Input files read: `rules.csv` , `requests.jsonl` , `expected_outputs.jsonl` , `decision.yaml` , `security_scan.json` , `load_test_results.json` , `canary_metrics_pass.json` , `canary_metrics_fail.json` , `deployment_state.json`

---

4. Section-by-Section Derivation

Section 1 – Decision Validation

Each of the 100 requests in `requests.jsonl` was evaluated against the seven rules in `rules.csv` in strict priority order (lowest priority number wins; first match exits).

Rule	Priority	Condition	Decision
H1	1	acuity_score ≥ 95	ED_NOW
H2	2	acuity_score ≥ 80	URGENT_REVIEW
H3	3	visit_type = telehealth AND acuity_score < 60	SELF_CARE
H4	4	location = RURAL AND acuity_score ≥ 60	CLINIC_REVIEW
H5	5	patient_group = immunocompromised AND acuity_score < 70	NURSE_CALL
H6	10	acuity_score < 60 AND symptom_severity = mild	SELF_CARE
H7	999	DEFAULT	ROUTINE_REVIEW

Result: **100/100 decisions matched** `expected_outputs.jsonl` (0 mismatches, 0.0% mismatch rate). A SHA-256 hash of `decision.yaml` is included as `config_hash` to provide configuration integrity verification. Since there were no mismatches, `decision_mismatches.jsonl` was not written.

Section 2 – Security Gate Evaluation

Policy sourced from `decision.yaml` : fail if `critical_vulns > 0` or `secrets_found > 0` . `security_scan.json` reported `critical_vulns=0` and `secrets_found=0` .

Result: **PASS** — "no critical vulnerabilities and no secrets detected"

Section 3 – Load Gate Evaluation

Thresholds from `decision.yaml` : `p95_latency_max=300ms` , `error_rate_max=1.0%` . Gate fails if either value meets or exceeds threshold. `load_test_results.json` reported `p95_ms=235` and `error_rate_pct=0.6` — both well under threshold.

Result: **PASS** — violations: []

Section 4 – Blue/Green Promotion Simulation

`deployment_state.json` shows `current_active=blue` , `inactive_version=green` . `canary_metrics_pass.json` reported `p95_ms=265` (< 300) and `error_rate_pct=0.9` (< 1.0) — both within policy. Canary gate passed, so the inactive (green) environment was promoted to active.

Result: **Promotion completed** — blue → green, rollback_triggered=false

Section 5 – Failure Scenario (Required for Principal Position)

`canary_metrics_fail.json` reported `p95_ms=390` (> 300) and `error_rate_pct=1.4` (> 1.0) — both exceed policy thresholds. Canary gate failed; promotion was blocked and rollback triggered automatically.

Result: **Canary FAIL** — rollback_triggered=true, reason=performance_threshold_exceeded

Section 6 – Release Policy Modeling

A policy definition was constructed from `decision.yaml` defaults with the governance requirement of a minimum 2-minute canary duration. All four gates (dataset, security, load, canary) were evaluated against this policy.

Gate	Result	Basis
dataset_gate	**PASS**	0 mismatches < 1.0% tolerance
security_gate	**PASS**	0 critical vulns, 0 secrets
load_gate	**PASS**	p95=235ms, error_rate=0.6%
canary_gate	**PASS**	p95=265ms, error_rate=0.9%

All gates passed → `promotion_allowed=true` , `overall_risk_level=LOW`

Section 7 – Incident & Risk Analysis

Given the hypothetical "canary error spike" incident (`error_rate=2.3%` , `p95=390ms` , `duration=3min`):

- **Classification:** `performance_regression` — both p95 latency and error rate exceeded policy thresholds.

- **Blast radius:** `10_percent_traffic` — the canary slice was 10% per the 90/10 split in `canary_metrics_fail.json` .
- **Rollback required:** `true` — any gate violation mandates rollback per policy.
- **Risk level:** `HIGH` — error rate (2.3%) exceeded 2× the 1.0% threshold.
- **Estimated user impact:** `moderate` — impact limited to the 10% canary slice over 3 minutes.

Section 8 – Observability & Error Budget

SLO targets defined as:

- Availability: 99.9%
- p95 latency: ≤ 300ms
- Error rate: ≤ 1.0%

Monthly error budget calculated as:

```
30 days × 24 hours × 60 minutes × (1 − 0.999) = 43,200 × 0.001 = 43.2 minutes
```

The observability plan also defines layered alert thresholds (warn at 0.5% error rate / 250ms p95, critical at policy threshold), a five-component monitoring strategy, three dashboards, and an on-call escalation policy.

Section 9 – Architecture & Governance

The architecture was derived from the deployment model described across `decision.yaml` , `deployment_state.json` , and the canary/promotion workflow:

Field	Value	Rationale
deployment_model	blue_green_with_canary	Active/inactive slots + 90/10 canary split
control_plane	policy_as_code	Rules and thresholds encoded in YAML/CSV
data_plane	stateless_service	No session affinity required for routing
governance_model	automated_gate_enforcement	All gates evaluated programmatically
gitops_alignment	true	Policy files are version-controlled artifacts
config_drift_risk	LOW	Deterministic rules with no runtime mutation

5. Final Summary

```
DATASET_MATCHES=100
SECURITY_GATE=PASS
LOAD_GATE=PASS
CANARY_GATE=PASS
PROMOTION_ALLOWED=true
ROLLBACK=false
OVERALL_RISK=LOW
```

6. Output File Index

File	Section	Description
`decision_run_report.json`	1	100 requests, 100 matches, 0 mismatches, config hash
`decision_mismatches.jsonl`	1	Per-case mismatches — written only if mismatches exist
`security_gate_evaluation.json`	2	Vulnerability scan gate result + reason
`load_gate_evaluation.json`	3	Performance load test gate result + violations
`deploy_report.json`	4	Blue→Green promotion result (canary pass scenario)
`deploy_report_rollback.json`	5	Blue→Green promotion result (canary fail / rollback scenario)
`release_policy_definition.json`	6	Policy thresholds and configuration
`release_policy_evaluation.json`	6	Gate evaluation results, promotion_allowed, risk level
`incident_analysis.json`	7	Canary error spike classification and risk

`observability_plan.json`	8	SLO definitions, alert thresholds, monitoring strategy
`error_budget_calculation.json`	8	Monthly error budget (43.2 min at 99.9% SLO)
`architecture_assessment.json`	9	Deployment model and governance structure
`summary.txt`	Final	Key=value summary for evaluator review